

Implementation Plan for the Healthonyx Full-Stack Platform

Overview: The Healthonyx platform is a comprehensive medical management system built with a Go backend, an Angular frontend, and a SQL database. It combines electronic medical records (EMR) features for doctors and patients with a knowledge base health monitor for internal documents. Below is a detailed breakdown of each major feature, including workflow, UI components, backend endpoints, database schema, and integration with notification APIs and LLM (Large Language Model) tools where applicable. Real-world enhancements (security, usability, scalability) are noted to ensure a production-grade design.

1. Document and Report Uploads (with AI Summaries)

Patients and doctors can upload external medical documents (e.g. lab results, imaging scans, referral letters) in PDF or image format to store in the system. These files become part of the patient's record and are visible to authorized users (the patient and their treating doctors). An AI summary of the content can be generated for quick insight.

- **Patient Uploads:** Patients have an interface to upload files such as external lab test results or home device readings ¹. Uploaded files may be marked as pending for doctor review if needed; once approved, they appear in the patient's record. Patients can see a list of all files they have uploaded, along with details like upload date and description ¹. For example, a patient who got blood work done at an outside lab can upload the PDF report into Healthonyx for their doctor to review.
- **Doctor Uploads:** Doctors can attach documents to a patient's chart, for example uploading a scanned consultation report or imaging result as part of their notes ². Each file is timestamped and labeled with who uploaded it. The system maintains a file upload history per patient so that nothing is overlooked ². This means both doctor- and patient-added documents (e.g. prior prescriptions, lab results) are tracked with dates and descriptions for comprehensive medical history.
- **UI Components:** Use an Angular Material file-picker (e.g. a **file upload** component with a drag-and-drop area or a simple `<input type="file">` field). When a file is selected, the user can add a short description (for context, like "Blood Test Results from 2025-08-15"). After upload, display the file in a list or card view on the patient's **Uploads** section and the doctor's view of that patient. Each entry should show file name/type, upload date, and uploader (patient or doctor). Provide a way to **preview** common file types: for images, show a thumbnail; for PDFs, either show a PDF viewer or offer a download link. For example, clicking a lab report might open an inline PDF viewer or trigger a download. This improves usability so doctors and patients can quickly access content.
- **Backend Endpoints & Storage:** Implement a Go API endpoint such as `POST /api/patients/{patientId}/files` to handle file uploads (accepting multipart form data). The backend will save the file to a storage location and create a database record for it ³. Files can be stored on the server's filesystem (e.g. under an `/uploads` directory) with the path saved in the database, or in a blob column in the database. A simple approach is storing files on disk and saving metadata (filename, path, etc.) in an **Attachment** table. The Attachment (or TestResult) table would include fields like: `file_id` (PK), `patient_id` (FK), `uploaded_by` (FK to User),

`date_uploaded`, `description`, and `file_path` or binary data ³. There can also be a field to link the file to a specific medical record or visit (`record_id`), if the upload is tied to a particular appointment ⁴. Provide a `GET /api/patients/{patientId}/files` endpoint to list files for that patient, and a `GET /api/files/{fileId}` to download or view a file (with proper authorization checks). The Go server can serve files directly or stream them from a cloud storage bucket (for scalability).

- **AI-Based Summaries:** As an optional enhancement, integrate an LLM service to generate a brief **summary of uploaded documents** for quick insight. For example, after a lab report PDF is uploaded, the system could extract the text (using OCR for images or reading PDF text) and then use an LLM to produce a plain-language summary of key findings (e.g. *"Elevated cholesterol noted, other values within normal range."*). This summary would be displayed to the viewer (doctor or patient) alongside the file. Implementation-wise, this could be done by sending the text to an **open-source model** (like a local instance of GPT4All or a HuggingFace Transformer summarization model) or an external API (like OpenAI GPT-3.5) if budget allows. Using open-source models with a library like **LangChain** can enable splitting the document into chunks and summarizing or answering questions about it. For more robust summaries, a **Retrieval-Augmented Generation (RAG)** approach could be used: e.g. indexing the document text with **Faiss** or a vector database and then having the LLM answer "What are the highlights of this report?". This ensures even long documents are handled. (If using Azure, one could leverage Azure Cognitive Search + OpenAI for this, but open-source tools avoid recurring costs.) The summary generation can be done asynchronously – for instance, a background Go routine or a separate Python service (triggered via an event or message queue) could compute the summary after the file is saved. The UI can show a spinner or "Summarizing report..." note, then display the summary when ready.
- **Security & Privacy:** Enforce strict access control on files – only the patient and authorized doctors (or admin) can access a given attachment. The download API should verify the requester's role and permissions. To protect sensitive data, files at rest can be encrypted or stored in a secure bucket. The system should also **validate file types and scan for malware** on upload (using an open-source antivirus like ClamAV) ⁵, since allowing arbitrary file uploads can introduce security risks. In a production setting, you might restrict file size and type (e.g. max 5 MB, only PDF/JPG/PNG), and use a virus scanner to prevent infected files from being stored. All file URLs should be protected (no direct public link unless a secure one-time token is provided, to prevent data leakage).

2. Prescription Module (E-Prescriptions & Reminders)

The platform will include a robust prescription management system. Doctors can write electronic prescriptions, update or revoke them as needed, and patients can view their prescriptions. The system can send medication reminders to patients (via email/SMS), which patients can opt into or out of individually. This improves medication adherence and keeps a clear record of treatments.

- **Doctor Workflow (Create/Update Prescription):** In the doctor's portal, when viewing a patient's record, the doctor can create a new prescription entry (for example, after a consultation). They will input details like the medication name, dosage, frequency (e.g. "twice daily"), duration (e.g. "for 2 weeks"), and any special instructions ("take with food", etc.). This prescription is saved to the database and linked to the patient's profile ⁶ ⁷. The doctor can later update the prescription (e.g. change the dosage) or revoke it (mark it as canceled) if necessary – for instance, if a medication is changed, the old prescription can be set to inactive. All changes should be tracked (with timestamps and who made the change) for audit purposes. If a prescription is revoked or updated, the patient should receive a notification of that change for transparency.

- **Patient View:** Patients can see a list of their **current and past prescriptions** in their dashboard or a dedicated “My Prescriptions” page. For each prescription, display the medication name, prescribed date, doctor’s name, dosage and frequency, start/end dates, and instructions. If a prescription is active (ongoing medication), highlight it and possibly show a progress indicator (e.g. “5 days remaining” based on duration). If a prescription was updated or canceled, indicate its status (e.g. “Updated on Jan 10” or “Revoked – no longer valid”). This helps patients keep track of what they should be taking and for how long. The patient can also download or print a prescription as a PDF – the system can generate a nicely formatted prescription document (including clinic info and doctor’s signature line) which can be used at pharmacies ⁶. *(In real-world integration, e-prescriptions might be sent directly to pharmacies, but for this scope a downloadable PDF is sufficient.)*
- **Database Model:** A **Prescription** table will store each prescription record. Key fields include: `prescription_id` (PK), `patient_id` (FK), `doctor_id` (FK), `date_prescribed`, `medication_name`, `dosage` (e.g. 10mg), `frequency` (e.g. “2x per day”), `duration` (e.g. 14 days), `instructions` (text), and `status` (active/updated/revoked) ⁷. We can also link a prescription to a specific visit record via `record_id` to know during which consultation it was prescribed ⁸. When a prescription is updated, one approach is to create a new record (with a new ID) and mark the old one as superseded; or simply update the fields and keep a change log. For simplicity, marking as “updated” with a note may suffice, but in production a history of prescription changes is useful.
- **Backend Endpoints:** Provide REST endpoints such as `POST /api/patients/{patientId}/prescriptions` for doctors to create a prescription (the doctor’s ID can be derived from the auth token or provided in the payload), `PUT /api/prescriptions/{id}` to modify or cancel a prescription, and `GET /api/patients/{patientId}/prescriptions` for a patient (or their doctor) to fetch the list. Only doctors (or admins) should be allowed to create/update prescriptions, and patients have read-only access to their own prescriptions. Business logic: when creating a prescription, the backend can automatically schedule reminder events (see next point) based on the frequency and duration.
- **Medication Reminders:** To improve adherence, the system can send automated reminders to the patient to take their medication as prescribed ⁹. For example, if a medication is “once daily at 8 AM for 14 days”, the system will schedule a daily notification at 8 AM for 14 days. We will use a background scheduler on the Go server (such as a cron job via the `robfig/cron` library) to handle these repeated notifications ¹⁰. When the doctor creates the prescription, they specify timing (or default to morning/evening based on frequency), and the backend sets up the schedule. The reminder message could be: “Reminder: Take your 10mg of MedicineX now (once daily as prescribed).” The platform will use either **email or SMS** based on patient preference to send these. We recommend using free-tier friendly services: e.g. **Twilio** for SMS (Twilio’s trial allows a limited number of messages to verified numbers, ~100 total messages, which is sufficient for development/testing) and **SendGrid or Mailgun** for emails (SendGrid’s free tier historically allowed ~100 emails/day) ¹¹. For example, Twilio’s Go SDK can send an SMS, and SendGrid provides an email API or SMTP relay ¹¹. All reminder content and scheduling should respect user consent – patients must opt-in to receive these (see **Notification Preferences** below) ¹². If a patient opts out, that particular reminder schedule is canceled.
- **Reminder Management:** Each prescription’s reminders should be individually controllable. In the UI, next to each active prescription, the patient might see a toggle like “Reminders On/Off” (for that medication). If they feel comfortable managing it without reminders, they can turn it off. This preference is stored (e.g. in a `ReminderPreference` linking the prescription and user). By default, when a doctor issues a new prescription, we assume reminders **on** (with the patient’s initial consent provided during onboarding), but the patient can disable it. The system also handles stopping reminders when the prescription’s course is completed or if the prescription is revoked early. Additionally, if the patient wants to receive reminders via one

channel only (say, email but not SMS), we allow that configuration in their settings (perhaps on a per-prescription basis or a global basis).

- **Notifications & Delivery:** The actual sending of emails/SMS for reminders will be handled by a background process. For scalability and reliability, we could queue these notifications in a table or in-memory job queue. For example, a **Notifications** table could log all scheduled and sent notifications (fields: `notification_id`, `user_id`, `type` [email/SMS], `message`, `scheduled_time`, `sent_time`, `status`) ¹³. A cron job can periodically check for due notifications and dispatch them. Using free-tier APIs: Twilio's trial (or low-cost pay-as-you-go after trial) and SendGrid/Mailgun SMTP can be configured in the system settings. Admins will input API keys for these services in an **Admin configuration page** ¹⁴ so the backend can authenticate to them. We will store these credentials securely (as env variables or encrypted in the DB).

3. Appointment Scheduling System

The appointment system allows doctors to set their availability and patients to book appointments with their preferred doctor or an alternate based on availability. The system prevents double-bookings and keeps both parties' calendars up to date. Confirmation and reminder notifications are sent for appointments.

- **Doctor Availability & Calendar:** Doctors can define their available time slots for patient appointments. For example, a doctor might set clinic hours or specific open slots (e.g. M/W/F 9am-12pm and 2pm-5pm in 30-minute blocks). In the doctor's interface, provide a **calendar view** (daily/weekly) where they can see their schedule and mark slots as available or blocked ¹⁵. They can also directly **schedule appointments** here (e.g. add a new appointment for Patient X on a given date/time). The calendar should visually show existing appointments to avoid conflicts. We can integrate a library like **FullCalendar** (which has an Angular wrapper and a free core) to display and manage events on the calendar UI ¹⁶. Alternatively, a simpler approach is a form where the doctor picks a date & time and assigns a patient to create an appointment entry ¹⁷, but a drag-and-drop calendar interface greatly improves usability.
- **Patient Booking Workflow:** Patients can request or directly book appointments through the portal ¹⁸. On the patient side, an "Book Appointment" feature lets them choose a doctor (from a list of doctors or just their primary doctor) and then see that doctor's available slots. For example, the patient selects Dr. Smith, then sees that Dr. Smith has openings on certain days/times and picks one that suits them. If using a real-time booking model, once the patient selects a slot, it's immediately reserved and an appointment is created. Optionally, if the clinic prefers to approve bookings, the request could go into a pending state for the doctor to confirm. For our scope, we assume direct booking for simplicity, with the system ensuring the slot was free. The patient will provide a reason or note for the visit (e.g. "Follow-up for blood pressure check"). After booking, the appointment appears on both the patient's and doctor's dashboards. If a patient's preferred doctor is unavailable in a timely manner, they could search other doctors by specialty or allow the system to suggest the **next available** slot with any doctor (this is a realistic enhancement in multi-doctor clinics).
- **Conflict Handling:** The backend logic for creating an appointment (`POST /api/appointments`) must ensure no double-booking. When a new appointment request comes in, the system checks if the doctor already has an appointment (or if the patient has another appointment) at that time. If a conflict is detected, it returns an error or suggests a different time. If the slot is free, it saves the appointment. The Appointment entity in the database would include: `appointment_id` (PK), `doctor_id` (FK), `patient_id` (FK), `datetime` (timestamp), `duration` (e.g. 30 minutes), `reason` (text), and `status` (scheduled, completed, canceled) ¹³ ¹⁹. The status can be updated by the doctor (e.g. mark as

completed after the visit, or canceled if the patient no-shows or reschedules). We may also track `created_by` (who booked it) and timestamps for auditing.

- **UI Components:** For doctors, provide a **schedule management UI** – possibly a calendar grid where they can click to open an “Add Appointment” dialog. For patients, provide a **doctor selection dropdown** and a **slot picker**. The slot picker could be a list of available times on the chosen date or a small calendar showing free days. We might reuse the same calendar component in read-only mode to display availability. Upon booking, show a confirmation screen with the appointment details. Both patient and doctor should be able to view upcoming appointments (in a list or calendar). Patients likely will have a simple list like “Upcoming Appointments” showing date, time, doctor name, and status. Doctors will see a list of their next appointments (with patient names and purposes) on their dashboard for quick access ²⁰.
- **Notifications (Confirmation & Reminders):** When an appointment is booked, the system should immediately send a **confirmation notification** to both the patient and doctor. This could be an email: e.g. “Appointment Confirmation: Dr. Smith with John Doe on 2026-03-15 at 3:00 PM.” If using SMS, a concise message like “Your appointment on Mar 15, 3:00 PM is confirmed.” For the doctor, the message might include patient name and contact info. These notifications use the same email/SMS infrastructure (SendGrid/Twilio) as other features. Additionally, the system will send **appointment reminders** to patients a day or two before the visit ²¹. Typically, an email reminder 24 hours prior is standard, possibly with any preparation instructions (e.g. “Remember to fast for 8 hours before your blood test”). SMS reminders can be sent the morning of the day before: “Reminder: You have an appointment with Dr. Smith tomorrow at 3:00 PM” ²¹. The admin can configure how early reminders are sent (perhaps a global setting, default 24h) ²². Like medication reminders, appointment reminders can respect patient preferences (some may opt for email only, etc.). The sending of these can be handled by the same scheduler system in the backend – e.g. when an appointment is created, schedule a one-time notification event 24 hours before the appointment time.
- **Rescheduling & Cancellation:** The system should allow either party (usually the patient calling the office, or doctor in emergencies) to cancel or reschedule an appointment. If a patient needs to cancel, they might do so through the portal (with a cutoff time to prevent last-minute cancellations online). Cancelling an appointment would update its status and free the slot for others. A cancellation notification can be sent. For rescheduling, one could cancel and create a new one, or if implementing directly, allow editing the date/time (with conflict check). Realistically, rescheduling might be handled by staff, but our system could allow patients to request a reschedule. These flows are considerations for a complete system.

4. Role-Based Dashboards (Patient & Doctor)

Both patients and doctors get personalized dashboards upon logging in, providing a snapshot of important information. The **Patient Dashboard** focuses on the individual's health progress and upcoming tasks, whereas the **Doctor Dashboard** gives an overview of their practice and patients under care. These dashboards use visual elements (charts, highlights) to make data easy to scan.

- **Patient Health Dashboard:** When a patient logs in, they are greeted with a quick overview of their health and activities. For example, the dashboard can show a **chart of recent health metrics** (if the patient logs data like blood pressure or if lab results are available) ²³. Trends can be visualized – e.g. a line chart of blood sugar readings over the past month. If the patient is on an ongoing treatment, display a **treatment progress** widget: for instance, “Medication adherence: 80% (8 of 10 doses taken on time)” or “Therapy sessions completed: 3/6”. If the system tracks something like steps or weight, those could be shown as well. Next, list **upcoming appointments** (with date, doctor, and countdown like “in 3 days”) ²⁴. Also show active **prescriptions** with a quick status (e.g. “Blood Pressure Medication – 5 days left” or “Ends on June

30"). If any new lab report or document has been uploaded (by doctor or patient), the dashboard could highlight "1 New Report Available" for the patient to view. A section for **health insights** can provide AI-driven or rule-based tips, such as "Your blood pressure has improved since your last visit, keep up the good work!" or "It's been over a year since your last full check-up ²⁵ ; consider scheduling an annual exam." This kind of gentle reminder can be derived from the data (here the rule is if >1 year since last physical, flag it ²⁵). Overall, the patient dashboard should make it easy for a busy patient to see what's next in their care and how they are doing in key areas. Visual cues (green/yellow/red status) can be used for things like adherence or lab result status.

- **Doctor's Dashboard:** Doctors see an overview of information that helps them manage their day and patient load. Key elements include: a **patient census** (how many patients they have under care, or how many are scheduled today) and an **appointments list for the day** ²⁶ . For example, "Today: 5 appointments (first at 9:00 AM, last at 4:30 PM)" and maybe a mini-list: 9:00 – John Doe, 10:00 – (free slot), 11:00 – Jane Smith, etc., each linkable to the patient's record ²⁶ . Unread or recent **messages** from patients are highlighted (e.g. "2 new patient messages") so the doctor can quickly jump to the messaging module ²⁷ . The dashboard can also show **alerts for patient care**: for instance, if a lab result came in that is critical or requires follow-up, an alert card might say "Lab Alert: John Doe's cholesterol is high – follow-up needed" ²⁸ . Similarly, if a patient hasn't responded to a follow-up or if a document in the knowledge base needs the doctor's attention, those can be flagged. A **workload chart** can be included to visualize the doctor's activities – for example, a bar chart of appointments per week or a pie chart of patients by condition (if such data is available) ²⁹ . This helps the doctor plan and manage time. If the doctor works in a team, their dashboard might also list any internal tasks or comments (e.g. "3 charts awaiting your sign-off"). Additionally, for doctors who are responsible for updating knowledge base documents (if any), there could be a small section showing, for example, "2 Medical Guidelines need review" as part of the integrated knowledge monitor.
- **LLM Summaries for Doctors:** To assist busy doctors, the dashboard (or patient list) can incorporate AI summaries. For example, next to each upcoming appointment, display a one-liner summary of the patient's recent history: "John Doe – Last visit 2 months ago for hypertension follow-up, latest labs show improved BP." This could be generated by analyzing the patient's record with an LLM (using the data in their notes and uploads). Similarly, if a patient uploaded a new document, the doctor's dashboard might show a snippet: "New lab result uploaded by Jane Smith: summary – **elevated cholesterol**" to draw the doctor's attention. These summaries can be precomputed whenever new data arrives. Technically, this could use the same summarization approach as document uploads (feeding relevant recent records to an LLM prompt). This feature is optional but demonstrates real-world usage of AI to save doctors time by condensing information.
- **UI and Components:** Both dashboards will be implemented as Angular components with a clean, widget-based layout. We can use **Angular Material cards** to separate sections (Appointments, Messages, Alerts, etc.) and **Angular charts** for graphs. For charts, open-source libraries like Chart.js (with the **ng2-charts** wrapper for Angular) or **NGX-Charts** (built on D3) can be used to create interactive charts ³⁰ . These are free and well-maintained. Each dashboard might have a 2-column responsive layout: e.g. left column has upcoming appointments and messages, right column has charts and alerts. Use icons and color codes (Material Design icons for calendar, pill, warning alerts, etc.) for visual clarity. Ensure the design is mobile-responsive so a doctor or patient on a phone can still see their key info (the layout might stack vertically on smaller screens).
- **Backend Data Aggregation:** The data shown on dashboards will be fetched via dedicated API endpoints that aggregate the needed info. For example, when the Angular app loads a doctor's dashboard, it might call `GET /api/doctor/{doctorId}/dashboard` which the backend implements to gather: count of patients, today's appointments, unread messages, alert counts,

etc., in one payload. This is more efficient than making separate calls for each item. Similarly, a `GET /api/patient/{patientId}/dashboard` could return their upcoming appointments, active prescriptions, recent metrics, etc. The Go backend will query the SQL database for these (possibly using JOINS or separate queries for each section). For visualizations like trends, the backend might pre-compute some stats (or the frontend can compute from raw data). Given that the data volume per user is not huge, on-the-fly aggregation is fine (e.g. count prescriptions, check latest lab values). If needed, we can add caching for expensive computations.

- **Health Insights & Alerts:** The dashboards will also integrate the “smart alerts” system. If certain rules or AI analyses find a potential issue (like a missing follow-up or overdue test), those alerts will show up on the appropriate dashboard ³¹ ³². For instance, if a patient hasn’t had a lab in a long time or a known condition isn’t addressed, the doctor dashboard could list that under “Attention needed” (and possibly the patient dashboard might nudge them to schedule something) ³³. These insights rely on data checks (some rule-based, some could be AI-based pattern recognition). We would implement a periodic job on the backend that scans records for such conditions (for example, daily check for any new critical lab results or any patients falling through the cracks) and then writes an entry to an Alerts table or directly notifies relevant users. This ensures that the dashboards always reflect up-to-date alerts to improve care quality.

5. Secure Messaging and Comments System

Healthonyx includes a secure messaging feature for direct communication between patients and doctors, as well as the ability to leave comments on specific records or documents. This allows asynchronous communication (similar to a built-in email/chat) and collaboration, while keeping all history within the platform for auditing and reference.

- **Patient-Doctor Direct Messaging:** Patients can send messages to their doctor through an in-app messaging interface ³⁴. For example, a patient who has a minor question after a visit (“Should I continue this medicine if I feel better?”) can send a message instead of calling the office. The doctor can reply when available, creating a chat-like conversation thread. This messaging works much like a secure email or chat: the patient selects the doctor (likely restricted to those who have seen them or their primary provider) and sends a text message; the doctor sees it in their inbox and responds ³⁴. The UI for this would be a **messaging center** where a user can see all conversations. We can list conversations by counterpart’s name (e.g. patient sees “Dr. Alice Smith”, doctor sees “John Doe”) and show an indicator for unread messages. Clicking a conversation opens the message thread (scrollable list of message bubbles, similar to chat apps). If both parties are online at the same time, we can enable real-time updates using WebSockets for an instant chat feel ³⁵. Otherwise, the system will simply fetch new messages periodically or on page load. We will use end-to-end encryption for message transit (HTTPS ensures encryption in transit, and if needed we can encrypt message content in the database for extra security since this is sensitive health communication).
- **Internal Comments on Records:** In addition to direct one-on-one messages, the platform supports commenting **within context of a record or document** ³⁶. For example, a doctor reviewing a patient’s lab result might leave a comment on that lab record: “This result looks improved, will discuss with patient next visit.” These comments could be visible to other providers in the team but not to the patient (if marked internal), or some could be shared with the patient. Similarly, a patient viewing a visit summary could comment: “I forgot, should I take this pill with food?” which would notify the doctor ³⁷. Each comment is attached to a specific entity – e.g. a **Visit record** (medical record entry) or an **uploaded document** – allowing asynchronous discussion about that item. The knowledge base documents can also have comment threads for collaborator doctors or admins to discuss content updates or clarifications ³⁸. This is useful for internal policy documents (e.g. an admin flags a guideline and writes a

comment “Is this information up to date?” to which a medical director can respond). The UI for comments would typically appear at the bottom of the record’s detail view: a list of existing comments (with user name and timestamp) and a textbox to add a new comment. It’s similar to how one might comment on a blog post or in Google Docs, but restricted to the relevant users.

- **Technology & Implementation:** For real-time messaging, we will integrate a **WebSocket** in the Go backend (using something like Gorilla WebSocket) ³⁵. When a user opens the messaging page, the frontend will establish a WebSocket connection to the server. This allows the server to push new messages instantly to the recipient’s browser (if online). If the user is offline, the message will be stored and delivered when they next connect (and we can also fall back to sending an email notification that “You have a new message on Healthonyx”). The messaging logic will be such that each message has sender, receiver, content, and timestamp. On the database side, we’ll have a **Message** table with fields: `message_id` (PK), `sender_id` (FK User), `receiver_id` (FK User), `content` (text), `timestamp`, and `is_read` (boolean) ³⁹. This covers direct two-party messages. For the comments attached to records or docs, we can use either the same table with additional fields (`context_type` and `context_id` to link to a PatientRecord, Prescription, or KnowledgeDoc) ⁴⁰, or a separate **Comments** table specifically for threaded comments. A simple approach: have a Comments table with `comment_id`, `author_id`, `text`, `timestamp`, `context_type` (enum: “Record”, “Document”, etc.), `context_id`, and maybe `visibility` (enum: “internal”, “shared with patient”). This way we store all comments in one place and query by context when displaying.
- **UI Components:** We will create an Angular **Messages component** for direct messaging. This might have two sub-components: a **ConversationList** (sidebar listing all conversation threads) and a **ChatWindow** (showing messages and input box for the selected conversation). Each message can be displayed in a chat bubble style (right-aligned for the current user’s messages, left-aligned for the other party). For comments on records, the component will be more compact: likely a simple list of comments under the relevant entry, and a small form to add a comment. We can use Angular’s two-way binding to update the comment list immediately when a user posts a new one (optimistically, while actually sending it to the server).
- **Notifications & Alerts:** The system will notify users of new messages or comments. For instance, if a patient sends a message to a doctor, the doctor should get an **in-app notification** (like a bell icon or a badge on the Messages menu showing an unread count) ⁴¹. If the message isn’t read in a timely manner, the system could also send an email: “You have a new message from Patient X on Healthonyx.” Similarly, if a doctor leaves a comment on a patient’s record and marks it for patient visibility, the patient could be notified to check that comment. We will implement a simple notification mechanism: when a new Message or Comment is inserted in the DB, create a notification entry for the recipient. The Angular app can poll the `/api/notifications` or use the WebSocket to receive a “new message” event. The UI would then display the indicator. Marking a message as read (when the user opens it) will update the `is_read` flag in DB to manage the unread count. For comment notifications, these might show up in a “Notifications” list in the UI as well (e.g. “Dr. Smith commented on your visit from Jan 5”). This ensures that even asynchronous comments get acknowledged. All these communications are stored permanently in the system, creating an audit trail and a richer patient record (e.g., a doctor can review message history to see what a patient asked between visits). Robust security (authentication, authorization) will wrap this feature: a user can only message their own doctor or patient, and cannot access others’ conversations. All communications involving personal health info remain within our encrypted database (unlike regular email, which is a privacy plus).

6. Knowledge Base Health Monitor (Internal Documents & AI Analysis)

This module manages the clinic's internal knowledge base – for example, medical guidelines, policies, or standard operating procedures – and continuously evaluates these documents for “health” factors like freshness, consistency, and accuracy. It provides analytics on these documents and uses AI to detect duplicates or contradictions, helping administrators keep the knowledge base up-to-date and reliable.

- **Knowledge Documents Overview:** The knowledge base consists of documents (articles) that staff or admins maintain. Each **KnowledgeDoc** entry might represent a protocol (e.g. “Diabetes Management Guidelines”) or a policy (“Clinic Sanitation Procedure”). The system will display these documents in a directory or list, likely categorized by topic. Each document has metadata including last updated date, author, etc., and importantly a **health score** that indicates its quality or freshness ⁴². The health score is a computed percentage or grade (e.g. 0-100 or A/B/C color codes) reflecting factors such as recency of update, usage frequency, and whether the content might conflict with other documents ⁴². For example, a document that hasn't been updated in over a year might have its score reduced due to staleness.
- **Versioning and Changelogs:** Every knowledge document is **version-controlled** ⁴³. When an admin or doctor edits a document, the system increments its version (v1.1, v1.2 for minor edits, v2.0 for major, etc.) and archives the old version. A **DocVersion** table stores the historical versions (snapshot of content, timestamp, editor) ⁴⁴. Users with appropriate access can view the change history or even compare differences between versions. This ensures that older guidelines are not lost and any changes are auditable. In the UI, when viewing a document, there could be a “Version History” button showing a list of past versions and who updated them when.
- **Freshness Tracking:** The system automatically flags **stale documents** that have not been updated within a certain threshold ⁴⁵. Administrators can configure this threshold in the admin settings (e.g. 6 months or 12 months) ⁴⁵. If a document exceeds that age without edits, it might be marked with an orange “Stale” label or lowered health score. The platform can generate tasks or reminders for admins to review those documents. For instance, if “Emergency Protocols” hasn't been reviewed in over a year, it shows up in a stale list, prompting an update to ensure accuracy. The system could even send a periodic report or notification listing approaching or overdue documents (e.g. quarterly review reminders) ⁴⁶.
- **Conflict & Similarity Detection:** Using AI, the knowledge monitor identifies when two or more documents have overlapping content or potential **contradictions** ⁴⁷. The platform will compute vector embeddings for each document's content (using NLP models) and perform similarity searches to find closely related documents ⁴⁸. For any pair that is very similar, it flags them for review – they might be duplicates or contain redundant info. More critically, if two guidelines conflict (say one document says “Drug X dosage is 5mg” and another says “Drug X dosage is 10mg”), the system should catch this. Implementation: we use a pre-trained **embedding model** (such as SentenceTransformers like MiniLM or mpnet) to convert documents into semantic vectors ⁴⁹ ⁵⁰. We can utilize an open-source library (e.g. run a Python service with HuggingFace, or use a Go library like kelindar's search which can run embeddings via llama.cpp in Go ⁵¹) to generate these vectors, avoiding external API costs. All document vectors are stored in a **vector index** – either in-memory with Faiss or in a small vector database like **Qdrant** (which has a free community edition and even a free cloud tier) ⁵². When a document is created or updated, its vector is upserted. Then we either continuously or on-demand run similarity queries: e.g. for each doc, find top 5 similar docs by cosine similarity. If the similarity score is above a threshold, mark them as possible duplicates. Those pairs get recorded in a **DocumentAlert** or **Conflict** table with details (docA, docB, similarity, issue status) ⁵³. For

contradiction detection, we go a step further: take two highly similar docs and feed their text (or key sections) to an LLM with a prompt like “Do these two documents have any conflicting statements? If so, what are they?”. An **open-source LLM** (like LLaMA-2 7B fine-tuned on QA) might do okay, but for more accuracy we might call a stronger model like GPT-4 via API ⁵⁴ ⁵⁵ . This can highlight the exact conflict (e.g. different dosage advice). The system can then show an alert: “Document A and Document B have conflicting instructions on drug dosage.” The admin can then resolve by editing or retiring one of them. This combination of vector similarity + LLM reasoning is essentially a form of **Retrieval-Augmented Generation (RAG)** used internally to maintain consistency ⁵⁶ .

- **Analytics Dashboard:** The knowledge base module provides an **analytics view** for admins (often part of the Admin Dashboard). This shows metrics like: total number of knowledge documents, average health score across all docs, percentage of documents updated in the last X months, number of documents flagged as stale, and number of conflict alerts open ⁵⁷ . For example, a widget might display “Docs updated in last 6 months: 85%” ⁵⁸ , and another might list the titles of the top 5 oldest documents or those with conflicts. Visual charts can illustrate the distribution of health scores (e.g. a pie chart of what fraction are green (>80), yellow, red (<50)) ⁵⁹ . Another chart might track over time how many documents were kept up-to-date (showing improvement or decline in maintenance). These analytics help administrators focus their efforts (perhaps the policy team needs to update those red-scored items).
- **UI for Knowledge Base:** In the Angular frontend, we’ll build a **Knowledge Base section** accessible to admin users (and possibly doctors, depending on policy). This will have: a **document list view** (sortable by title, date, score, etc.), a **document editor** view (rich text editor for creating/updating content, with versioning controls), and a **dashboard** view for analytics. Each document entry in the list can show a colored health score icon (like a small colored dot or percentage) for quick scanning ⁴² . It will also show last updated date (with perhaps a warning icon if beyond threshold). Admins can filter to “Show stale documents” or “Show conflicts”. There might be a separate tab or page for “Alerts/Conflicts” listing all identified issues (with links to the docs in question). For implementing the editor, we could use a third-party rich text editor component (to allow formatting content). Edits when saved via `PUT /api/knowledge/docs/{id}` will create a new version entry. The UI for comparing versions could be basic (show two texts side by side or simply indicate that a doc was updated from v1.0 to v1.1). The comment feature on documents (for internal discussion) as mentioned earlier would also surface here.
- **Integration with Clinical Side:** Although the knowledge base is mainly for internal use (doctors and admins), we may allow patients to access certain educational articles. For example, general health articles (if included in the knowledge base) could be made visible to patients (perhaps marked as public). This could be a future enhancement for patient education. The system design will consider a `visibility` property on KnowledgeDoc if needed to distinguish internal vs public documents.
- **LLM and Azure Option:** To summarize the AI integration, we have two modes: open-source or Azure. Open-source uses local embeddings (no cost) and possibly smaller local LLMs. Azure’s alternative would use **Cognitive Search** (Free tier allows ~50 MB index, which is plenty for text of a few thousand docs) to handle vector storage and search ⁶⁰ , and **Azure OpenAI** to do the heavy NLP like checking contradictions or answering queries ⁶¹ . The Azure approach could be seamless but introduces cost – however, it’s modest. For example, generating embeddings with Azure’s service costs on the order of **\$0.00002 per 1K tokens** (essentially \$0.02 per million tokens) and querying GPT-3.5 is about **\$0.002 per 1K tokens** ⁶² . So even if we did a few hundred comparisons or Q&A per month, the cost would be only a few dollars, likely within free credits or negligible ⁶² . We will document both options: the free open-source pipeline using LangChain+Faiss (or Qdrant) with a local model, versus the Azure cognitive services pipeline. This way, the deployment can choose the fully free route or a low-cost improved accuracy route. In

either case, the system greatly enhances knowledge maintenance by proactively spotting outdated or conflicting information, which is crucial in a medical setting for patient safety.

7. Notification Preferences and Delivery Controls

Managing how and when users receive notifications is important for user satisfaction and compliance. Healthonyx will allow patients (and other users) to customize their notification settings – for example, turning certain reminders on or off, or choosing the channel (email or SMS) – on a granular level. The admin can oversee the notification system status and logs of messages sent.

- **Patient Notification Settings:** Each patient can access a **Notification Preferences** page in the portal. This page lists the different types of notifications the system sends: Medication Reminders, Appointment Reminders, New Lab Result Alerts, General Announcements, etc. For each category (or even each specific item like each prescription), the patient can choose to enable or disable notifications, and possibly choose the medium (email and/or SMS). For example, a patient might toggle OFF reminders for a particular prescription if they no longer need daily nudges, but leave other prescriptions' reminders ON. They might prefer appointment reminders via email only, so they uncheck SMS for that category. These preferences give patients control and help avoid spamming or annoyance (important for adherence – too many messages can cause users to ignore them). By default, we assume critical notifications (like a critical lab alert) will be sent unless explicitly opted-out, for safety. The UI for this can use toggles (using Angular Material slide toggles) next to each option, and perhaps a dropdown for channel selection. Changes are saved via an endpoint like `PUT /api/patients/{id}/preferences/notifications`.
- **Doctor/Admin Notification Settings:** Similarly, doctors might have preferences (e.g. a doctor could opt to get SMS alerts if a patient's lab is critical, or turn off email for certain less urgent system notifications). Admins might configure to get emails on system errors, etc. We will include these settings for completeness, though the prompt focuses on patient toggles. Essentially, every user can have a profile of how they get notified for different event types.
- **Backend Implementation:** We'll model notification preferences in the database. One approach is a **UserPreferences** table, which can include JSON or separate columns for each category of notification. A more normalized approach: a **NotificationPreference** table with entries like (user_id, notification_type, channel_email_bool, channel_sms_bool, enabled_bool). For instance: (user123, "med_reminder_prescription456", email=true, sms=false, enabled=true) could represent that user123 wants email but not SMS for prescription 456 reminders. Storing by specific item (like each prescription ID) could get large; instead, perhaps we store general categories (e.g. med_reminders in general) and by default send all active meds if that category is on. However, since the user specifically asked for "on/off per prescription or report", we might allow very granular control. We could implement that by inheriting the general setting but also having an override list. To keep it simple: each prescription could have a boolean flag in the Prescription table for reminders (which the patient UI toggles), and each appointment could have a flag if a patient turned off that reminder. In code, before sending a reminder, the system checks the corresponding flag or preference entry.
- **Admin Panel – Notifications Monitoring:** In the admin interface, there will be a section showing the status of the notification system. This could include: a log of recent notifications sent (e.g. "Jan 5 10:00 – SMS to +15551234 for prescription reminder to user JohnDoe – Sent ☑"), any failures (if an email failed to send, log the error), and counts of pending scheduled messages. Essentially, an admin **notifications log** viewer. This can be backed by the Notifications table that was described under prescriptions/appointments, which logs each outgoing message ¹³. The admin can filter by date or type to troubleshoot (e.g. to confirm that yesterday all appointment reminders went out). Additionally, the admin could have controls to globally

disable certain notifications in emergencies (for example, turn off all reminders with one switch, or adjust the sending times). The admin might also set the default times for daily reminders here (e.g. send medication reminders at 8 AM local time – this could be configurable) ¹⁴ .

- **Opt-In/Opt-Out Compliance:** The system will ensure that patients have given consent for notifications (likely as part of onboarding or in profile settings) ¹² . A patient can opt out entirely from all automated notifications if they prefer (though we will warn them that they might miss important health reminders). Each notification sent should include a line like “Don’t want these reminders? Update your preferences in Healthonyx.” to comply with good practice. We’ll also honor things like time-of-day preferences if needed (e.g. not sending non-urgent messages at 2 AM). In the US, for SMS, compliance with TCPA would require explicit opt-in; since this is a closed system for patients, we gather that at sign-up.
- **Technical Delivery Notes:** We use the same infrastructure (Twilio, SendGrid/Mailgun) to send out notifications. These APIs typically return status codes; the backend should record if sends succeed or fail. For emails, using an SMTP server or API, we’d catch exceptions and mark notification as failed (so that maybe it can retry or alert admin). Twilio SMS API also gives delivery status callbacks if configured. For simplicity, we may not implement full callback handling on free tier, but we can log that the request was accepted. Admin can see those statuses in the log. Over time, if the volume grows, switching to a background job queue system like a RabbitMQ or a cloud function for sending might be prudent – but at our scale (with free tier usage), a synchronous send in the scheduler loop is fine.

8. Admin Panel and System Management

Healthonyx provides an **Admin Panel** for system administrators to manage users, configurations, and view high-level analytics and logs. The admin role has the widest access and can oversee both the clinical and knowledge base aspects to ensure everything runs smoothly. Key features of the admin panel include user/role management, dashboards for system metrics, knowledge base health monitoring (described above), audit logs, and global settings.

- **User & Role Management:** Admins can create, edit, or deactivate user accounts for doctors and patients ⁶³ . This includes onboarding new doctors (assigning them login credentials and possibly a department or specialization) and patients (though patients might also self-register if allowed, the admin can manage those accounts). The UI will likely have a **User Management** table listing all users with their role, and actions to add or disable accounts. For example, an admin adding a new doctor will fill a form with name, email, temporary password, role=Doctor, and any doctor-specific fields like license number or specialty. Under the hood, we have a **User** table (with common fields user_id, name, email, password_hash, role, etc.) and separate **Doctor** and **Patient** profile tables linked by user_id ⁶⁴ ⁶⁵ . The admin interface abstracts this – e.g., selecting role “Doctor” in the form will show the extra fields needed for the Doctor table. Role-Based Access Control (RBAC) is enforced so that only admins can reach these endpoints ⁶⁶ . If needed, admins can also promote a user’s role (e.g. make a doctor also an admin) or adjust permissions (like toggling whether a patient can do certain things). However, the roles are likely fixed tiers (Patient, Doctor, Admin).
- **System Dashboard (Admin Analytics):** Upon logging in, an admin sees a high-level **system overview dashboard** ⁶⁷ ⁶⁸ . This may include: number of active patients and doctors, number of new sign-ups this week, total appointments in the last month, total prescriptions issued, etc. Essentially, it aggregates usage data. It can also show **system health** info: if we have monitoring endpoints on the backend, we could surface things like server uptime or database status here ⁶⁹ . But primarily, it’s for usage metrics and key performance indicators for the clinic. For example, charts: “New Patients per Month” (bar chart), “Appointments by Department” (pie chart if multiple specialties), or “Average appointments per day”. These give administrators insight into

how the platform is being used and the clinic's operational data ⁷⁰. We will use the same chart libraries (Chart.js/NGX-Charts) to render these on the admin frontend. The data comes from various tables (counts, group by queries, etc.). Potential metrics: *Patient Engagement* – e.g. % of patients using the portal (could be measured by login or uploads), *Medication Adherence Average* – if we track how many reminders were acknowledged (an advanced metric), *Most common medications prescribed*, *No-show rate* – if appointments have a status field for completed vs no-show. These are nice-to-have analytics that can be added incrementally.

- **Knowledge Base Health:** The admin panel integrates the knowledge base dashboard described in Feature 6. Admins can navigate to a **Knowledge Base** section to see the health metrics of internal documents ⁵⁸. This includes the charts and lists of stale or conflicting documents. For instance, an admin might see “5 documents need review” and click to see which those are. They can then assign someone to update them or do it themselves. This part of the admin panel ensures the informational content remains reliable and is a differentiator of Healthonyx (combining EMR with knowledge management).
- **Audit Logs:** Given the sensitivity of medical data, every significant action should be logged. The admin panel will have an **Audit Log** view ⁷¹. This could be a table of records like: timestamp, user, action, target. For example: “2026-02-01 10:23, Dr. Jones, Edited, Prescription #123 (dosage changed)” or “2026-02-02 15:45, Patient JaneDoe, Uploaded, file LabResult_ABC.pdf”. The log helps in security audits and tracking any suspicious behavior. Admins can filter logs by user or date to investigate issues (e.g. if a patient claims something was altered, the logs show which account did what). To implement this, the backend will create log entries on critical endpoints – either manually or via an interceptor/middleware that captures changes. We might have a simple **Log table** (id, timestamp, user_id, action_type, entity_type, entity_id, description). The admin UI for this is mostly for viewing/searching these logs.
- **Global Configuration:** A **Settings** section in the admin panel allows configuration of system-wide settings ¹⁴. Examples: the email/SMS gateway credentials (Twilio API keys, SendGrid API key) can be entered and updated here (these would be stored securely, possibly encrypted, and only loaded by the backend). Other settings: the “document staleness threshold” in days for the knowledge base ¹⁴ (e.g. admin sets it to 180 days or 365 days as policy), default reminder lead time for appointments (e.g. 24h), toggling patient self-service features (maybe an admin checkbox “Allow patients to self-register” or “Allow patients to book appointments directly” – turning these off could hide those features platform-wide if a clinic doesn't want them). The UI would present these in a form, and updating them would call an endpoint like `PUT /api/admin/settings`. The backend would then use these configurations (possibly reloading into memory or reading from DB as needed). Some settings might require a restart (if we don't build a dynamic reload), but we will focus on ones that can be applied at runtime.
- **Role-Based Views and Security:** The Angular app will route users to the appropriate dashboard based on role after login. Admins have access to everything (or at least to admin-specific routes). We will implement guard services in Angular to prevent unauthorized access (e.g. a patient should never be able to manually navigate to an admin URL). Similarly on the backend, endpoints are protected by middleware that checks the user's role from their JWT or session. All data access is scoped: doctors can only retrieve data for their patients; patients only their own data; admins can get anything but typically use admin-specific endpoints. This RBAC system is crucial and was planned in design ⁷². We will likely have a middleware that checks something like `if user.role !== requiredRole: return 403`. For multi-role (like a doctor who is also admin), we'll allow them admin access if role includes admin. Passwords are stored hashed (e.g. bcrypt) and secure practices (like using HTTPS, secure cookies for session if not using JWT, etc.) are followed. The admin panel will also allow initiating **password resets** or account recovery for users who need assistance.
- **Deployment and Monitoring:** While not exactly part of the panel UI, a production-grade plan should mention that we'd deploy the Angular frontend perhaps on a static hosting (Netlify or

Vercel) and the Go backend on a free-tier cloud (Heroku, Render, etc.), using a free SQL database (like ElephantsQL for Postgres). Admins (or devops personnel) would monitor logs on these platforms for any runtime issues ⁷³. In future, containerizing and scaling out can be done as needed, but initially the focus is on a functional free-tier deployment. The admin panel's presence ensures that a single clinic's instance of Healthonyx can be managed without direct database edits – most control is available through the UI, making it a self-sufficient system.

User Flow Example: Busy Doctor and Patient Interaction

To illustrate how all these features come together, here's a structured scenario of a busy patient and doctor using the Healthonyx platform in a real-world style workflow:

- 1. Patient Onboards & Checks Dashboard:** Jane Doe, a patient, logs into Healthonyx. On her **Patient Dashboard**, she immediately sees she has an upcoming appointment with Dr. Smith tomorrow, and a reminder that her "Blood Pressure Medication – 5 days left" on the current prescription. She also notices a chart showing her blood pressure readings trending downward – a positive sign. An alert on the dashboard mentions "It's been over 1 year since your last full blood panel." Seeing this, Jane decides to message the doctor about scheduling a lab test.
- 2. Patient Books an Appointment:** Before messaging, Jane clicks "Book Appointment." She selects Dr. Alice Smith (her primary doctor) and sees available time slots. She chooses a slot next week Wednesday at 10:00 AM for a lab test follow-up. The system instantly creates the appointment (no conflict as Dr. Smith had that slot free) and shows a confirmation. Jane's dashboard now lists the new appointment. Dr. Smith also gets a notification of this booking on her side.
- 3. Doctor's Daily Review:** Dr. Alice Smith logs in at the start of her day. On her **Doctor Dashboard**, she sees "Today: 4 Appointments" – the first one with John (9am), etc., and one open slot at 11am. She sees a red alert icon indicating one of her patients has a critical lab result. She clicks it and it shows "John Doe's cholesterol is critically high – follow up needed." She also notices she has 1 unread patient message.
- 4. Doctor Checks Messages:** Dr. Smith opens the **Messages** section and sees a conversation from Jane Doe (from overnight). Jane's message reads: "Doctor, should I schedule a new lab test since it's been a year? Also, I'm often forgetting my evening dose – any tips?" Dr. Smith replies via the chat: "Yes, let's do a blood panel. I see you already booked an appointment, that's perfect. For the evening dose, I recommend setting an alarm; I'll also have the system continue sending reminders. We can discuss more in person." Jane, who is online, sees the doctor's reply pop up in her chat window and feels reassured.
- 5. Doctor Views Patient Record:** Before Jane's visit, Dr. Smith clicks on Jane Doe's profile from her patient list. She reviews Jane's record: past visit notes, current prescriptions, and recent uploads. Jane had uploaded an external **lab report PDF** last week from a cardiologist visit. The system has an AI-generated summary for it: "**Summary of External Report:** ECG is normal, cholesterol slightly elevated, recommendation to increase exercise." This saves Dr. Smith time – she quickly grasps the content without reading the full report. Dr. Smith opens the file to verify a detail, then writes a quick comment on that report: "Noted. We'll recheck cholesterol in our lab." This comment is visible to Jane, who gets notified of it.
- 6. Prescription Management:** During Jane's upcoming visit (now happening in the clinic), Dr. Smith conducts the exam and updates Jane's record in Healthonyx. She navigates to the **Prescriptions** module in Jane's profile and updates the blood pressure medication dosage (lowering it). She marks the previous prescription as superseded. The system logs this change and immediately sends Jane an app notification (and an email) that "Your prescription for MedicineX was updated

by Dr. Smith.” Dr. Smith also adds a new prescription for a cholesterol-lowering drug, with instructions and a schedule. She sets the medication reminder for this to daily at 8 AM.

7. **Automated Reminders:** That evening, Healthonyx’s scheduler service creates the first 8 AM reminder for the next day’s dose of the new medication. Jane has opted in for SMS reminders, so the next morning she gets a text: “Reminder: Take your CholesterolMed 20mg now, as prescribed by Dr. Smith.” Jane, finding these helpful, leaves the reminders on. Over the next week, she diligently takes her medication when reminded. If she wanted, she could toggle off the reminder from her portal, but she keeps it on.
8. **Appointment Reminder:** The day before Jane’s lab appointment, Healthonyx automatically sends her an email reminder: “You have an appointment with Dr. Smith tomorrow at 10:00 AM. Please remember to fast for 8 hours before your blood test.” Dr. Smith’s office configured this message in the appointment details or it was included as default text for lab appointments. Jane confirms she’ll be there.
9. **Post-Appointment Follow-up:** After the appointment and lab work, Dr. Smith receives the lab results (perhaps as a PDF from the lab). She uses Healthonyx to **upload the lab report** to Jane’s record. The system again generates a summary: “Lab results: Cholesterol improved to 180 mg/dL, all other values normal.” Dr. Smith reviews it and is pleased. The system’s intelligence also checks: since cholesterol was a concern and now improved with medication, it resolves the earlier alert for Jane’s high cholesterol follow-up. Jane later logs in and sees the new lab report (with summary) in her uploads. She also sees a health insight on her dashboard: “Your cholesterol has decreased to a normal range – great progress!”
10. **Knowledge Base Update:** Meanwhile, the clinic’s administrator reviews the **Knowledge Base Health Monitor**. The dashboard shows 2 documents with low health scores. One is an old “COVID-19 Policy” last updated 3 years ago – clearly stale. The admin opens it, updates it with the latest guidelines, and saves a new version. This automatically resets its staleness flag and improves its health score. The second alert is a conflict: two documents on “Hypertension Treatment” seem to overlap. The admin sees the AI’s note that Document A recommends a certain drug dosage that Document B does not mention. After consulting with the medical director, they decide to merge the documents. The admin uses the platform to mark Document B as archived and updates Document A to include any missing info. The conflict alert is then marked resolved.
11. **Admin Oversight:** Finally, the admin checks the **Admin Panel** logs and settings. They verify that all email reminders have been sent (seeing a log list of messages and their sent status). They also adjust a setting to shorten appointment reminder lead time to 12 hours (maybe patients said 24h was too early to remember). The admin adds a new doctor account for a specialist joining next week via the user management page. Confident that the system is running well (the System Dashboard shows healthy activity and no errors), the admin logs out.

In this end-to-end flow, we saw the patient and doctor seamlessly use the platform’s features: uploading and summarizing documents, scheduling and notifications, prescription updates with reminders, secure messaging, dashboards providing insights, and the admin ensuring the knowledge base and system settings stay optimal. This demonstrates how Healthonyx can be used in a realistic busy clinic scenario to enhance communication, adherence, and knowledge management all in one integrated system.

Sources: The design and features above incorporate and extend the specifications from the provided technical guide and notes, ensuring a production-ready approach with modern tech stack tools and free-tier services for notifications and AI. Key references from the technical guide include functionality confirmations [6](#) [18](#) [23](#) , design justifications for reminders and messaging [74](#) [34](#) , and the use of open-source/Azure AI for document analysis [54](#) [61](#) , among others. This comprehensive plan aims to

cover all requested features with practical implementation details, providing a clear roadmap for developing the Healthonyx platform.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

Full-Stack Medical Monitoring and Document

Health Platform – Technical & Feature Guide.pdf

file:///file_00000000f20c71f594be605596fe3069